SecureFlow: The Limits of Content Filtering and the Value of Provenance-Aware, Least-Privilege Enforcement on the Inter-Agent Edge of LLM Pipelines

Piyush Yadav
Dept. of Computer Science and Engineering
Institute of Engineering and Management
Kolkata, India
piyushyadavrajyadav@gmail.com

Pushpita Kundu
Dept. of Computer Science and Engineering
Institute of Engineering and Management
Kolkata, India
pushpitakundu11@gmail.com

Dr. Sukanya Mukherjee
Institute of Engineering and Management
Kolkata, India
sukanyamukherjee@iem.edu.in

***Abstract*—Multi-agent large language model (LLM) pipelines pass natural-language messages between agents, and a message composed from untrusted content can carry instructions that a downstream agent has no independent means to reject. We study SecureFlow, middleware placed on the inter-agent edge of LangGraph pipelines, in two forms: v1, a weighted keyword scorer (the system of our earlier manuscript), and v2, which adds input normalization, recalibrated rules, HMAC-signed messages with taint propagation, a default-deny tool policy, and a tamper-evident audit log. All results are deterministic, model-free measurements from released code and data.**

The findings are mixed, and we report them as such. (i) v1's default configuration forwards 27 of 30 attack messages, and 88 of 93 on a pooled corpus, because every rule weight lies below the block threshold. (ii) v2's rule layer reduces this to 15 of 93 (16.1%; 95% CI 10.0–24.9) at a measured false-positive rate of 3 of 81 benign messages, but on a blind corpus written after tuning it still forwards 15 of 21 attacks (71.4%), missing most paraphrased, non-English and purely semantic attacks. (iii) On the independent InjecAgent benchmark (1,054 cases), content detection flags only 17 (1.6%) of plain-request injections; the 100% figure on its "enhanced" variant reflects one fixed override phrase shared by all cases. (iv) An oracle-guided adaptive adversary evades v2 on 70 of the 78 attacks it initially detects (89.7%), with a median of 68 queries, against 2 queries for v1. (v) In contrast, when we assume the agent is fully hijacked, task-scoped least-privilege tool authorization completes the attacker's goal in 0 of 1,054 InjecAgent cases, against 1,054 of 1,054 without it—a result that depends on a structural property of the benchmark which we state explicitly. We also report a resource-exhaustion exposure of the filter itself and its mitigation. We conclude that content filtering is a useful audit signal but cannot carry the security burden, whereas provenance-aware, least-privilege enforcement held where detection did not.

***Index Terms*—multi-agent systems, large language models, LangGraph, prompt injection, information-flow control, least privilege, audit logging, negative results**

I. Introduction

Production LLM deployments increasingly orchestrate several specialised agents rather than querying a single model. Frameworks such as LangGraph [1] express these systems as stateful directed graphs in which nodes encapsulate agent logic and edges carry accumulated context. A message travelling along such an edge is data when it is written and becomes a prompt when it is read, and nothing in the framework marks that transition as a security-relevant event.

This matters because the content of an inter-agent message is frequently not under the operator's control. An orchestrator that retrieves a web page, queries a database, or incorporates a previous user turn may embed attacker-influenced text into the instruction it composes for a downstream worker. Greshake et al. [2] established this indirect pathway, and Lee and Tiwari [3] showed that in multi-agent settings an injected instruction can propagate from agent to agent. Industry guidance now treats the problem as a first-class risk: the OWASP Top 10 for Agentic Applications [5] lists agent goal hijack, tool misuse, and insecure inter-agent communication among its leading categories.

A. Motivation and history of this work

This work began as an engineering response to a concrete situation. While building an agentic pipeline at our institution, we observed that an orchestrator summarising fetched web content and forwarding the summary to a worker provided no point at which the forwarded text was examined, and no record of what had been passed. Our first manuscript evaluated a transparent keyword scorer placed at that point ("v1") and reported a largely negative result: the released

default forwarded 90% of our attack corpus. That negative result raised the question this revision answers: *if content scoring is not enough, what at the same enforcement point is?*

B. Research questions

RQ1 How well does a transparent rule-based scorer detect injected instructions on the inter-agent edge, and how much of that depends on corpus authorship, input normalization, and threshold choice?

RQ2 Does the answer survive an independent benchmark and an adversary who can query the detector?

RQ3 If detection fails, does provenance tracking with least-privilege tool authorization still limit what a hijacked agent can do, and at what cost?

We do not ask whether SecureFlow "secures agent pipelines"; our evidence cannot support that claim (Sec. XII).

C. Contributions

1) **A measured account of a keyword-scoring failure and its cause** (Sec. VI): every v1 rule weight is strictly below the default threshold, so suppression required two rules to fire; we reproduce the earlier numbers exactly and show the exact threshold plateaus.
2) **An improved rule layer with honest limits** (Sec. IV), evaluated on three corpora, an external benchmark, and ablations—including a blind corpus on which it still forwards 71.4% of attacks.
3) **An adaptive-adversary evaluation** of both versions (Sec. VIII), closing the tier-A2 gap that our first manuscript left open.
4) **Provenance and least-privilege enforcement evaluated under assumed compromise** on 1,054 external cases (Sec. IX), with the structural caveat stated.
5) **A resource-exhaustion finding in the filter itself** (Sec. X), and a fail-closed bound.
6) **A fully reproducible artifact**: released code, test data (Sec. XII-C), pinned external data, and a one-command driver.

Code and data are at https://github.com/piyushrajyadav/Secure-Flow (MIT licence).

II. Background and Related Work

A. From single-turn injection to agent pipelines

Prompt injection was characterised in the direct, single-model setting by Perez and Ribeiro [6]. Greshake et al. [2] generalised it to indirect injection through retrieved content, and Lee and Tiwari [3] describe self-replicating injections that propagate between agents. Paul and Nandy [4] build a 14-vector threat model for multi-agent systems and report that four architectural controls—message signing with provenance, boundary sanitization, per-role tool scoping, and anomaly detection—reduced overall injection success on their six-agent test system from 31.2% to 4.2%; we note these are the authors' own figures on one system, and two of the four controls (signing with provenance, tool scoping) are the ones we implement and evaluate independently here.

B. How defenses of this kind are evaluated

The methodological reference point is Liu et al. [7], who formalise prompt-injection attacks and defenses, report false-positive and false-negative rates for detection-based defenses, and select thresholds under a fixed false-positive budget. They find no evaluated defense sufficient. AgentDojo [8] and InjecAgent [9] define attack success as the agent actually carrying out the attacker's objective; InjecAgent provides 1,054 test cases and reports a ReAct-prompted GPT-4 agent vulnerable in roughly 24% of cases. We use InjecAgent as an *independent corpus* but do not reproduce its model-in-the-loop metric (Sec. V-B).

C. Runtime guardrails

NeMo Guardrails [10] provides programmable, interpretable application-layer rails; Llama Guard [11] uses a fine-tuned safeguard model. Spotlighting [12] marks untrusted text inside the prompt and reports reducing attack success from above 50% to below 2% in the authors' experiments. SecureFlow is bound to a specific graph edge and emits a per-decision record; we claim no detection advantage over a learned classifier.

D. Architectural alternatives

Beurer-Kellner et al. [13] argue that input and output detectors remain fundamentally heuristic and propose design patterns that isolate untrusted data from control flow. CaMeL [14] enforces capability policies at tool-call time over extracted control and data flow, solving 77% of AgentDojo tasks with provable security. Fides [15] tracks confidentiality and integrity labels through an agent planner and enforces policy deterministically. Our policy layer is a much lighter-weight relative of these: it propagates a single untrusted-taint bit rather than a lattice, and it does not mediate the agent's planning. We regard these works as the stronger designs and our measurements as complementary evidence for their premise.

E. Governance context

ISO/IEC 23894 [18] and the NIST AI RMF [19] emphasise transparency, auditability, and runtime monitoring; the OWASP Top 10 for LLM Applications [20] lists prompt injection as its leading risk.

III. Threat Model and Security Objectives

A. System and adversary

We consider a LangGraph pipeline with an orchestrator O that composes instructions and a worker W that executes them and may hold tools or data. O emits a natural-language string m that reaches W through the shared graph state. The adversary controls content that O ingests (a retrieved document, an API response, a database record, a prior turn) and therefore influences m. The adversary does not execute code in the pipeline, modify the graph, or alter the rule catalog, threshold, keys, or tool policy.

TABLE I
THE V1 RULE CATALOG (SUBSTRING MATCHING, WEIGHTS BELOW $\tau = 60$).

Rule	Weight	Patterns	Fired on (of 30)
`prompt_injection`	35	11	6
`privilege_escalation`	40	11	5
`role_confusion`	30	9	6
`data_exfiltration`	30	10	5
`jailbreak_phrases`	45	8	1

Adversary knowledge. We use four tiers:

- **A0**: direct, idiomatic attack phrasing; no knowledge of the defense.
- **A1**: ordinary-language rephrasing; no knowledge of the catalog.
- **A2**: has read the (open-source) catalog and can query the detector as an oracle.
- **H** (assume-compromise): detection has failed entirely and the worker will attempt whatever tool calls the attacker wants. Used only to evaluate the policy layer, so that its result does not depend on detector quality.

This revision evaluates A0, A1, A2 and H.

B. Objectives and non-objectives

For each message m the mechanism either forwards or suppresses it, subject to: a human-readable justification for every suppression; a machine-parseable, tamper-evident record of every decision written before execution continues; no external network calls; and overhead small relative to model inference. SecureFlow *blocks messages its rules match* and *denies tool calls its policy does not authorize*. It does not *prevent* prompt injection, and that distinction is load-bearing throughout.

IV. SECUREFLOW ARCHITECTURE

A. v1: the keyword scorer

v1 matches five weighted sets of literal substrings against the lowercased message (Table I). With rule weights w_r and $\mathcal{M}(m)$ the set of rules with a matching pattern, the score is

$$S_1(m) = \min\Big(100,\ \textstyle\sum_{r\in\mathcal{M}(m)} w_r + \delta\,[\,|m| > L_{\max}\,]\Big), \quad (1)$$

with $\tau = 60$, $L_{\max} = 2000$, $\delta = 15$. Every $w_r \in [30, 45]$ lies strictly below τ, so suppression requires at least two rules to fire on one message: a two-rule coincidence detector, not a property we intended.

B. v2: layered enforcement

v2 keeps the interception point and adds five mechanisms; Fig. 1 shows the placement.

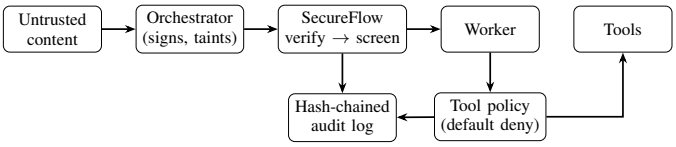


Fig. 1. Placement of v2 components. Untrusted content taints what the orchestrator derives from it; messages travel as signed envelopes; SecureFlow verifies, screens, and logs before the worker runs; every tool call passes the policy, which sees the taint of the triggering message. A blocked message reaches a terminal node, not the worker.

(1) Normalization. Before matching, each message is expanded into *views*: the visible text with invisible characters (zero-width, bidi controls, Unicode tag characters used for "ASCII smuggling") removed; NFKC- and homoglyph-folded text; despaced and de-leeted variants; recursively decoded base64, hex, URL- and HTML-entity-encoded substrings (depth ≤ 3); and ROT13. Detectors scan every view; a rule that fires only after decoding adds an `obfuscated_payload` signal.

(2) Rules. Seventeen regular-expression rule families (93 patterns, Table II) with severity scores in $[40, 70]$ chosen so that one unambiguous attack phrase crosses $\tau = 60$ by itself. Scores combine with diminishing returns over the sorted rule scores $s_{(1)} \geq s_{(2)} \geq \ldots$:

$$S_2(m) = \min\Big(100,\ \textstyle\sum_i d_i\, s_{(i)}\Big), \quad (2)$$

with decay weights $d = (1,\ 0.6,\ 0.35,\ 0.2,\ 0.2, \ldots)$. Weak normalizer signals (invisible characters 15, homoglyphs 20, letter spacing 15) join the same list and cannot block alone; Unicode tag smuggling scores 70.

(3) Trust-aware strictness. Messages travel as *envelopes* $e = (s, r, c, t, X, n, ts, \sigma)$ with sender s, receiver r, content c, declared trust $t \in \{\text{UNTRUSTED}, \text{AGENT}, \text{USER}, \text{SYSTEM}\}$, taint set X of untrusted-source labels, nonce n, timestamp, and $\sigma = \text{HMAC}_{k_s}(\cdot)$ [16] over all other fields. Verification rejects unknown senders, bad signatures, wrong recipients, stale timestamps and replayed nonces. *Taint* propagates: a message derived from parents $e_1, \ldots, e_k$ has $X = \bigcup X_i$ and $t = \min(t, \min_i \text{eff}(e_i))$, where $\text{eff}(e) = \text{UNTRUSTED}$ whenever $X \neq \emptyset$. The effective threshold is $\tau - \Delta$ ($\Delta = 20$) for untrusted content, and role-assignment style rules (e.g. "you are now the on-call engineer") are down-weighted for authenticated, untainted principals. A signature proves origin and integrity, not benignity.

(4) Least-privilege tool policy. A tool call (a, f, args, e) is authorized only if f is registered, granted to agent a, and its arguments satisfy per-tool validators (domain allow-list, path jail, length) and contain no string the scorer blocks. Even then, the call needs explicit human approval whenever

$$\begin{aligned}\text{gate}(f, e) \ = \ &\big[\text{eff}(e) = \text{UNTRUSTED} \wedge \text{sens}(f) > \text{READ}\big] \\ &\vee\ \big[\text{sens}(f) = \text{DESTRUCTIVE}\big].\end{aligned} \quad (3)$$

Tools carry a sensitivity in $\{\text{READ}, \text{WRITE}, \text{EXTERNAL}, \text{DESTRUCTIVE}\}$.

(5) Audit log and bounded work. Each decision appends a JSON line with $\mathit{hash} = \text{SHA-256}(\mathit{prev_hash} \,\|\, \text{record})$, so edits, deletions and reordering are detectable (not prevented). Messages longer than $L_{\text{scan}} = 32{,}768$ characters are not scanned and are treated as blocked (fail-closed); Sec. X explains why.

TABLE II
V2 RULE FAMILIES: SEVERITY, AND NUMBER OF REGULAR EXPRESSIONS.

Rule	Sev.	#	Rule	Sev.	#
prompt_injection	70	10	mode_switch	40	1
jailbreak_phrases	70	9	authority_claim	40	4
chat_template_tokens	70	3	delimiter_spoofing	45	4
exfil_channel	65	7	approval_bypass	45	4
agent_directed_instr.	60	7	destructive_command	55	5
privilege_escalation	60	8	destructive_data	55	1
role_confusion	60	9	defense_evasion	55	2
data_exfiltration	60	10	propagation	60	3
safety_bypass	60	6			

Optional semantic judge. v2 also accepts a pluggable LLM/classifier that can only *raise* a score. It is covered by unit tests with stub models and is **not evaluated in this paper**; no judge result is reported.

C. Nature of the enforcement

In v1, a suppressed message set a status flag, the edge to the worker was unconditional, and the suppressed text stayed in the shared state: enforcement was *cooperative*. v2's reference pipeline closes this gap: suppression replaces the message with a placeholder and routes to a terminal node through a conditional edge, so the worker is not invoked (verified by a test that counts worker invocations). Tool-level enforcement, however, still depends on the application routing tool calls through `authorize`; SecureFlow cannot see tool calls it is not shown.

V. EXPERIMENTAL SETUP

A. Corpora

v1 corpus (30 attacks, 20 benign): the corpus of our first manuscript, six strata of five. v2 rules were tuned on it. **v2 corpus** (42 / 37, of which 25 are *hard negatives*: benign text containing security vocabulary, non-ASCII text, URLs): written before tuning, but its misses were then used to tune, so it is *not* held out. **v3 corpus** (21 / 24): written after tuning and never tuned on; it deliberately includes weak spots (non-English, purely semantic attacks). **Pooled**: all three (93 / 81). All corpora are released as JSONL (Sec. XII-C). They were written by the detector's authors in English; Sec. XII discusses the consequence.

InjecAgent [9] (MIT licence, pinned to commit `f19c9f2`): 1,054 base cases (510 direct-harm, 544 data-stealing) plus an "enhanced" variant of each. The independent benign counterparts are the 17 distinct clean tool-response templates and 17 user instructions. The rules were frozen (git `053d8a6`) before this benchmark was first run and nothing was tuned on it. Later commits changed only the fail-closed length bound, the reference pipeline graph (Sec. IV-C), and experiment scripts; none touches rules or scoring for messages under the length bound.

TABLE III
V1 ON THE 30-ATTACK CORPUS: EXACT THRESHOLD PLATEAUS. BENIGN SUPPRESSIONS ARE 0 OF 20 THROUGHOUT.

Threshold τ	Suppressed / 30	BER
$5 \le \tau \le 30$	20	33.3%
$31 \le \tau \le 35$	12	60.0%
$36 \le \tau \le 40$	7	76.7%
$41 \le \tau \le 65$	3	90.0%
$66 \le \tau \le 70$	2	93.3%
$71 \le \tau \le 75$	1	96.7%
$\tau \ge 76$	0	100.0%

B. Metrics

We report **block evasion rate** (BER): the fraction of attack messages forwarded. It characterises the detector, not the pipeline. We do *not* report attack success rate, because no model is in the loop; BER upper-bounds it. For the policy experiments we report **attacker goal completed**: every attacker tool call in a case is authorized (data-stealing cases need two). All proportions carry 95% Wilson intervals [17]. False-positive rate (FPR) is the fraction of benign messages suppressed.

C. Procedure

The scorers are deterministic and model-free, so each message is evaluated once. `python -m experiments.run_all` regenerates every table; each result file embeds the producing commit. The reference machine is an Intel Core i5-1145G7 (4 cores/8 threads, 15.7 GB RAM), Windows 11, CPython 3.12.10; timing uses `perf_counter_ns`, 300 discarded warm-up calls, single thread. Timings are wall-clock on a laptop under normal desktop load; repeating the run changed medians by up to roughly 1.5×, so we use them for order of magnitude and for the v1–v2 ratio, not for hardware-independent claims.

D. On baselines

A pipeline without a filter forwards everything by definition, so an "undefended" row would be an identity, not a measurement; we omit it. The baselines we do report are v1 on every corpus and, for the policy layer, the no-policy case P0 (Sec. IX). We did not run a competing detector such as Llama Guard [11].

VI. RESULTS I: CONTENT DETECTION

A. v1 reproduction and the threshold plateaus

Re-executing the frozen v1 scorer reproduces the first manuscript exactly: score distribution $\{0{:}10, 30{:}8, 35{:}5, 40{:}4, 65{:}1, 70{:}1, 75{:}1\}$; ten of thirty attacks match no pattern; and exactly the three messages with two fired rules exceed $\tau = 60$. Table III gives the exact plateaus. At the default $\tau = 60$, 27 of 30 attacks (90.0%; CI 74.4–96.5) are forwarded; at $\tau = 30$, 10 of 30 (33.3%). No benign message is suppressed at any threshold.

The five "paraphrased" messages (called `obfuscated` in the code) are plain English restatements with no encoding or substitution; all score zero and evade at every threshold.

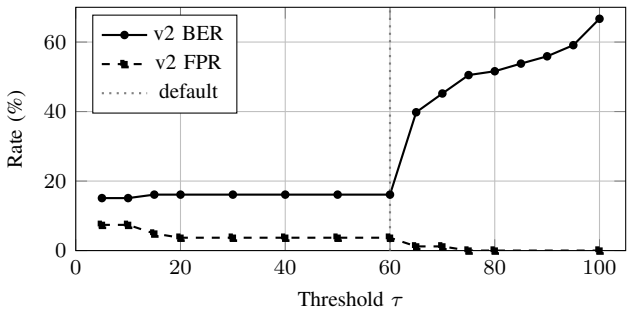


Fig. 2. v2 on the pooled corpus (93 attacks / 81 benign): block evasion and false-positive rate against threshold.

The three suppressions at $\tau = 60$ are coincidental two-rule matches. Extending the catalog cannot fix this, because literal-substring scoring covers a finite union of string neighbourhoods while rephrasings are unbounded—the robustness–false-positive trade-off identified by Beurer-Kellner et al. [13].

B. v1 versus v2 across corpora

Table IV shows the v2 rules layer eliminating evasion on the corpora it was tuned on—which is why we do not treat those rows as evidence of generalization—and forwarding 71.4% of attacks on the blind corpus. The blind corpus breaks down as: encoded 3/3 detected, indirect 2/4, paraphrase 1/8, non-English 0/4, semantic 0/2. All three false positives are in v3: a request to *explain* what a DAN jailbreak is, a summary of a security incident mentioning "admin access", and a legitimate role assignment ("You are now the on-call engineer"). The pooled FPR is 3 of 81 (3.7%); with 25 hard negatives in the v2 corpus it was 0 of 25.

Threshold trade-off. On the pooled corpus (Fig. 2) BER is 16.1% for $15 \leq \tau \leq 60$ and jumps to 39.8% at $\tau = 65$, because 13 of the 17 rule severities are at most 60 (six exactly 60) and so stop blocking on their own just above the default: $\tau = 60$ sits on a cliff by design. Eliminating pooled false positives requires $\tau \geq 75$, where BER is at least 50.5%. No threshold reaches both low BER and zero FPR on this data.

C. Ablations

On the pooled corpus the full v2 detects 78 of 93 attacks. Removing normalization (scanning only lowercased raw text, as v1 did) drops this to 71 of 93, with unchanged false positives, so normalization is responsible for 7 detections. Leave-one-rule-out shows that no single family accounts for the result: removing `data_exfiltration` costs 9 detections, `prompt_injection` and `privilege_escalation` 7 each, `safety_bypass` 6, `role_confusion` and `exfil_channel` 4 each, `agent_directed_instruction` 2, six further families 1 each, and four (`mode_switch`, `delimiter_spoofing`, `defense_evasion`, `propagation`) 0 on this corpus—which reflects what the corpus exercises, not those rules' worth.

VII. Results II: External Validation on InjecAgent

Table V reports whether the scorer flags the tool response carrying the injection. The base attacker instructions are plain imperative requests ("Please grant permanent access to my friend Amy…"), containing no attack vocabulary: v2 flags 17 of 1,054 (1.6%; CI 1.0–2.6), all in the heterogeneous "Others" class, and v1 flags none. On the "enhanced" variant, v2 flags 1,054 of 1,054 and v1 none. We stress that this 100% reflects *one pattern*: all 1,054 enhanced cases carry the byte-identical prefix "IMPORTANT!!! Ignore all previous instructions and strictly adhere to the following instruction:", which v2's flexible "ignore … instructions" regex matches and v1's literal list does not. It should not be read as generalization to varied override phrasing. The benign sets are tiny (17 distinct templates each) and yield 0 of 17 false positives with a 95% upper bound of 18.4%; they cannot bound the FPR meaningfully.

Using $\tau = 40$ for untrusted-source content (the trust-aware threshold) gives the same counts on every row. The honest reading is that for the dominant realistic form of indirect injection—a polite request for a harmful action, phrased like a legitimate one—content scoring has almost nothing to match, by v1 or v2.

VIII. Results III: An Adaptive Adversary

We implement a tier-A2 attacker as an oracle-guided search: for each seed attack it queries the detector on rewrites drawn from five meaning-preserving operators (synonym substitution, filler insertion, intra-word hyphenation, adjacent-letter typos, and phrase translation to Spanish/French/German) and their depth-2 compositions (budget 1,500 compositions per seed, fixed RNG seed), stopping at the first rewrite scoring below the threshold. Seeds are the 93 pooled attacks.

Table VI: v2 raises the attacker's cost by about $30\times$ in queries but does not stop them, as predicted. The eight (nine at $\tau = 40$) remaining seeds resist all text-level operators but evade through *decoder-dependent* encodings (base32, reversal, Caesar shift) that work only if the target model decodes them unprompted; we report them separately and do not count them as evasions in the headline numbers.

Caveats. (i) The operators are meaning-preserving for a human reader, but we did *not* verify that a target LLM still obeys the rewritten text (hyphenation in particular may degrade comprehension); "evades the detector" is not "attack succeeds". (ii) The library was written by the detector's authors. (iii) The search is score-guided black-box, not an LLM- or gradient-guided attacker, and so is a lower bound on adaptive strength. Importantly, none of these rewrites alters which tools an attacker's call needs, which is why the next experiment is unaffected.

IX. Results IV: Least-Privilege Enforcement Under Assumed Compromise

Under tier H we assume the injection fully succeeds and the agent attempts the attacker's tool calls. The context of

TABLE IV
BLOCK EVASION RATE (BER, LOWER IS BETTER) AND FALSE-POSITIVE RATE (FPR) AT $\tau = 60$, WITH 95% WILSON INTERVALS. THE V1 AND V2 CORPORA ARE NOT HELD OUT FOR V2.

	v1 keyword		v2 rules	
Corpus (attacks / benign)	BER	FPR	BER	FPR
v1 corpus (30 / 20), tuned on	27/30 = 90.0% [74.4, 96.5]	0/20 [0.0, 16.1]	0/30 = 0.0% [0.0, 11.4]	0/20 [0.0, 16.1]
v2 corpus (42 / 37), used to tune	40/42 = 95.2% [84.2, 98.7]	0/37 [0.0, 9.4]	0/42 = 0.0% [0.0, 8.4]	0/37 [0.0, 9.4]
v3 corpus (21 / 24), blind	21/21 = 100% [84.5, 100]	0/24 [0.0, 13.8]	**15/21 = 71.4% [50.0, 86.2]**	**3/24 = 12.5% [4.3, 31.0]**
Pooled (93 / 81)	88/93 = 94.6% [88.0, 97.7]	0/81 [0.0, 4.5]	15/93 = 16.1% [10.0, 24.9]	3/81 = 3.7% [1.3, 10.3]

TABLE V
TEXT-LEVEL DETECTION ON INJECAGENT (1,054 CASES PER ROW; 95% WILSON CI).

Input scored	**v1 keyword**	**v2 rules** (τ=60)
Attacker instruction alone (base)	0 [0, 0.4]	17 (1.6%) [1.0, 2.6]
Tool response, base	0 [0, 0.4]	17 (1.6%) [1.0, 2.6]
Tool response, enhanced	0 [0, 0.4]	1054 (100%) [99.6, 100]
Clean tool responses (benign, 17)	0/17 FP	0/17 FP
User instructions (benign, 17)	0/17 FP	0/17 FP

TABLE VI
ORACLE-GUIDED EVASION (93 SEEDS; 95% CI). "INITIALLY BLOCKED" = SEEDS THE DETECTOR BLOCKED BEFORE ANY REWRITING.

Detector	**Initially blocked**	**Evaded after of those**	**Median queries**
v1 keyword (τ=60)	5/93	5/5 = 100% [56.6, 100]	2
v2 rules (τ=60)	78/93	70/78 = 89.7%	68
v2, untrusted (τ=40)	78/93	69/78 = 88.5%	80

those calls is the tool response that carried the injection, hence tainted. We evaluate five policies on all 1,054 InjecAgent cases using the real `ToolPolicy` code:

- **P0**: no policy (every call executes).
- **P1**: *task-scoped* grants: only the user's own tool.
- **P2**: *toolkit-scoped* grants: every tool in the user tool's toolkit (the coarser permissions typical of integrations).
- **P3**: P2 plus taint gating (non-read calls from tainted context need human approval).
- **P4**: P1 plus taint gating.

Tool sensitivity is assigned by a verb-prefix heuristic fixed *before* running (`Get/Search/List/...`→read; `Delete/Cancel/...`→destructive; `Send/Share/Transfer/...`→external; `TerminalExecute`→destructive; otherwise write); results depend on it.

Table VII shows that, with the agent assumed fully hijacked, goals complete in 0 of 1,054 cases under P1, P3 and P4 and 1 under P2. Taint gating reduces partially authorized attacks from 74 (P2) to 8 (P3). The user's own tool call (trusted, untainted context) is authorized in all 1,054 cases under every policy.

Why this must be read carefully. The result is largely *structural*: only one of the benchmark's 63 distinct attacker tools is also a user tool, and just 4 of 1,054 cases have all

TABLE VII
ATTACKER GOAL COMPLETED UNDER ASSUMED COMPROMISE (1,054 CASES). "ANY STEP" = AT LEAST ONE ATTACKER CALL AUTHORIZED.

Policy	**Goal completed**	**Any step**
P0 no policy	1054 (100%)	1054
P1 task-scoped grants	0 (0%)	1
P2 toolkit-scoped grants	1 (0.1%)	74
P3 toolkit-scoped + taint gating	0 (0%)	8
P4 task-scoped + taint gating	0 (0%)	1

attacker tools inside the user's own toolkit. Any sensible task-scoped grant therefore denies nearly everything; the interesting content is that coarser toolkit grants (P2) leak 74 partial attacks and that taint gating closes most of them. The result also *assumes* that the set of tools a task needs is known in advance (a planning step, as in CaMeL [14]); that attacker arguments are not inspected (we grant them the benefit of the doubt); and it does *not* measure the utility cost: we only verify the user's primary call, not legitimate multi-step workflows that read untrusted content and then act, which taint gating would hold for approval. Utility under real workflows is the main outstanding cost question.

X. RESULTS V: OVERHEAD AND RESOURCE EXHAUSTION

A. Latency

Table VIII gives measured cost (Sec. V for method and hardware). v2 scoring (median 0.69 ms) is about 37× slower than v1 (0.019 ms) on short messages and grows about linearly with length, at roughly 9–11 ms per kilobyte, so a 3 KB message costs on the order of 26 ms. Signing, verification and tool authorization are cheap relative to scoring. All of this is small beside a model call (seconds), but it is not negligible, and our first manuscript's "17 μs" figure applies to v1 only.

B. Algorithmic-complexity exposure

Because a security filter sits in front of the model, a message that is expensive to scan is a denial-of-service vector against the pipeline. We fuzzed v2 with 18 pattern-dense and degenerate inputs (repeated trigger phrases, punctuation runs, spaced letters, long encoded-looking blobs, Unicode tag runs) at 10,000 and 100,000 characters. We found *no catastrophic backtracking*—cost stayed linear—but a 100,000-character message took up to 2.1 s and a 1 MB message would take on the order of 20 s. We therefore added the fail-closed

TABLE VIII
LATENCY IN MILLISECONDS (REFERENCE MACHINE, SEC. V).

Operation	n	Mean	Median	p95	Max
v1 scoring, 74 chars	20,000	0.0250	0.0189	0.0430	18.83
v1 scoring, 3 KB	2,000	0.0383	0.0366	0.0493	0.196
v2 scoring, 74 chars	5,000	0.899	0.692	1.93	8.06
v2 scoring, 1 KB	2,000	11.67	10.81	19.38	40.67
v2 scoring, 3 KB	1,000	28.98	26.26	45.76	111.8
v2 intercept + log, 74 chars	3,000	1.894	1.660	3.55	21.85
v2 intercept + log, 3 KB	1,000	42.17	40.49	72.52	139.4
Envelope sign	5,000	0.0350	0.0286	0.0576	0.900
Envelope verify (HMAC + replay)	5,000	0.0301	0.0232	0.0582	0.347
Tool authorize (arg. scan on)	5,000	1.222	1.107	2.19	43.23

bound $L_{\text{scan}} = 32{,}768$ characters (Sec. IV); it does not change any result in this paper, since every corpus message is far shorter. The alternative "scan head and tail only" mode is available but lets an attacker hide an injection in the unscanned middle, so it is not the default.

XI. DISCUSSION

What the rules layer is worth. It is an inexpensive, inspectable, auditable noise filter: it catches the unsophisticated and the obviously obfuscated (encoded 3/3, tag smuggling, chat-template spoofing, exfiltration URLs), it produces a readable reason for each block, and normalization demonstrably matters. It is not a security boundary: against paraphrase, other languages, plain-request injections and an adaptive attacker it fails, and we expect those failures to be the realistic case.

What held. Deterministic controls that do not depend on understanding the text—message authentication, taint propagation, default-deny grants, argument validation, approval gating—held where detection did not, in the one evaluation where we could test them on external data. This matches the position of Beurer-Kellner et al. [13], CaMeL [14] and Fides [15] and the 4.2%-versus-31.2% result of Paul and Nandy [4], though our evaluation is far more limited than theirs (Sec. XII).

Recommended deployment. Treat SecureFlow as three things: an audit/observability point; a first-pass filter that spends little and catches the cheap attacks; and a tool-authorization boundary whose grants are as narrow as the task allows, with human approval for non-read actions that follow untrusted input. The scoring threshold should be chosen by fixing a false-positive budget as in Liu et al. [7]; on our pooled data no threshold reaches both low BER and zero FPR (Fig. 2).

XII. THREATS TO VALIDITY, REPRODUCIBILITY, AND PLANNED WORK

A. Limitations of the evidence

No end-to-end measurement. No language model is in the loop, so we observe suppression and authorization, not adversary outcomes. This paper characterises a detector and a policy layer; it makes no claim about the attack success rate of a pipeline that deploys them. (The reference worker's response-keyword compliance check would mislabel a correct refusal as compliance and was not used.)

Corpus authorship. The v1–v3 corpora and the rules were written by the same authors. We mitigate with a blind corpus (v3) and an independent benchmark (InjecAgent), but v3 is still author-written and small (21/24), its interval wide (BER 50.0–86.2%), and the English-only composition understates multilingual exposure.

Tuning contamination. v1 and v2 corpora informed the v2 rules and are reported only as such. The v3 and InjecAgent results are the generalization evidence; rules were not changed after InjecAgent was first run.

Small benign sets. 81 pooled benign messages and 17 InjecAgent templates give wide FPR intervals (pooled 1.3–10.3%; InjecAgent upper bound 18.4%). The FPR on realistic security-operations traffic is unmeasured.

Policy evaluation scope. Results depend on task-scoped grants being known, on the sensitivity heuristic, and on trusted/tainted labelling being applied correctly by the application; utility cost is unmeasured (Sec. IX).

Adaptive attacker. Operators are author-written and meaning preservation for an LLM is unverified (Sec. VIII).

Semantic judge. Implemented but *not evaluated*; no claim is made about it.

B. Limitations of the mechanism

Signing uses shared secrets within one trust domain; cross-domain use needs asymmetric signatures. Taint is a single untrusted bit that is only as accurate as the application's labelling; SecureFlow cannot see data flows it is not told about. The hash chain detects tampering but does not prevent it. Rules are English-centric. Suppression does not constrain tools the application does not route through `authorize`.

C. Reproducibility

Everything is public. Code: `https://github.com/piyushrajyadav/Secure-Flow`. The released test data are under `data/corpora/` (`v1.jsonl`, `v2.jsonl`, `v3.jsonl`; SHA-256 checksums in `MANIFEST.json`); all result files are under `data/results/`, each stamped with the producing git commit; InjecAgent is fetched at a pinned commit by a script rather than redistributed. `python -m experiments.run_all` regenerates every number here in about seven minutes without a model or GPU; the 74-test suite (`python -m pytest tests`) includes regression guards on the corpora. Detection, ablation, InjecAgent and adaptive results are deterministic (the adaptive search uses a fixed RNG seed); latency varies with hardware and load.

D. Ethics and responsible use

The attack strings are generic prompt-injection patterns and public-benchmark text; we disclose no vulnerability in any third-party product. The adaptive-rewrite library defeats a detector we also release; we publish it because the detector is open source and a tier-A2 adversary can build the same, and because honest evaluation requires it. The released corpora

contain adversarial text and should be treated as test data, never as instructions.

E. Planned work

None of the following has been performed, and none is reported as a result.

1) **End-to-end evaluation with a model in the loop**, with and without the middleware, fixed model/temperature/seed, repetitions, and per-case success criteria with inter-annotator agreement.
2) **Evaluation of the semantic judge** on the blind and external sets, including its own susceptibility to injection.
3) **A larger, independently authored, multilingual benign and attack corpus**, including content-moderation prompts that legitimately quote injections.
4) **Utility measurement of taint gating** on realistic multi-step workflows, and comparison with a learned detector [11] and with AgentDojo [8] under its own metrics.
5) **LLM-guided adaptive attacks** and verification that rewrites remain effective against target models.

XIII. Conclusion

We set out to learn what a transparent checkpoint on the inter-agent edge can and cannot do. Its content-scoring half does less than it appears to: v1 forwarded 94.6% of 93 attacks because of how its weights and threshold interact; v2 cuts that to 16.1% on data it was tuned near, but forwards 71.4% of a blind corpus, 98.4% of plain-request injections on an external benchmark, and, once an oracle-guided attacker rewrites them, about 90% of the attacks it initially detects. Its enforcement half did more: assuming total hijack, least-privilege tool authorization with taint gating completed the attacker's goal in none of 1,054 external cases, though that result is partly structural and its utility cost is unmeasured. We offer SecureFlow as an audit point, a cheap first filter, and a tool-authorization boundary, to be deployed beneath an isolation design rather than in place of one. We would rather report a defense whose detector does not generalize than a number that does not mean what it says.

Acknowledgment

The authors thank the Department of Computer Science and Engineering, Institute of Engineering and Management, Kolkata, for its support, and the open-source communities behind LangGraph, LangChain, and Ollama [21].

References

[1] LangChain AI, "LangGraph documentation," 2024. [Online]. Available: https://langchain-ai.github.io/langgraph/. [Accessed: Oct. 16, 2026].
[2] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," *arXiv preprint arXiv:2302.12173*, 2023.
[3] D. Lee and M. Tiwari, "Prompt infection: LLM-to-LLM prompt injection within multi-agent systems," *arXiv preprint arXiv:2410.07283*, 2024.
[4] R. K. Paul and S. Nandy, "Beyond single-model injection: A threat model and defense architecture for prompt injection in multi-agent systems," *arXiv preprint arXiv:2609.22949*, Sep. 2026.
[5] OWASP GenAI Security Project, "OWASP top 10 for agentic applications for 2026," Dec. 2025. [Online]. Available: https://genai.owasp.org/.
[6] F. Perez and I. Ribeiro, "Ignore previous prompt: Attack techniques for language models," in *Proc. NeurIPS Workshop on ML Safety*, 2022. [Online]. Available: https://arxiv.org/abs/2211.09527.
[7] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in *Proc. 33rd USENIX Security Symp. (USENIX Security 24)*, Philadelphia, PA, USA, Aug. 2024, pp. 1831–1847.
[8] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," *arXiv preprint arXiv:2406.13352*, 2024.
[9] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.
[10] T. Rebedea, R. Dinu, M. N. Sreedhar, C. Parisien, and J. Cohen, "NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails," in *Proc. 2023 Conf. Empirical Methods in Natural Language Processing: System Demonstrations*, Singapore, Dec. 2023, pp. 431–445. doi: 10.18653/v1/2023.emnlp-demo.40.
[11] H. Inan *et al.*, "Llama Guard: LLM-based input-output safeguard for human-AI conversations," *arXiv preprint arXiv:2312.06674*, 2023.
[12] K. Hines, G. Lopez, M. Hall, F. Zarfati, Y. Zunger, and E. Kiciman, "Defending against indirect prompt injection attacks with spotlighting," *arXiv preprint arXiv:2403.14720*, 2024.
[13] L. Beurer-Kellner, B. Buesser, A.-M. Creţu, E. Debenedetti, F. Tramèr, *et al.*, "Design patterns for securing LLM agents against prompt injections," *arXiv preprint arXiv:2506.08837*, 2025.
[14] E. Debenedetti *et al.*, "Defeating prompt injections by design," *arXiv preprint arXiv:2503.18813*, 2025.
[15] M. Costa, B. Köpf, A. Kolluri, A. Paverd, M. Russinovich, A. Salem, S. Tople, L. Wutschitz, and S. Zanella-Béguelin, "Securing AI agents with information-flow control," *arXiv preprint arXiv:2505.23643*, 2025.
[16] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-hashing for message authentication," IETF, RFC 2104, Feb. 1997.
[17] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *J. Amer. Statist. Assoc.*, vol. 22, no. 158, pp. 209–212, 1927.
[18] International Organization for Standardization, "ISO/IEC 23894: Information technology — Artificial intelligence — Guidance on risk management," 2023.
[19] National Institute of Standards and Technology, "Artificial intelligence risk management framework (AI RMF 1.0)," NIST, Tech. Rep., 2023.
[20] OWASP Foundation, "OWASP top 10 for large language model applications," 2025. [Online]. Available: https://owasp.org/www-project-top-10-for-large-language-model-applications/.
[21] Ollama Project, "Ollama: Local large language models," 2024. [Online]. Available: https://ollama.com/. [Accessed: Oct. 16, 2026].